

Tervezési minták:

A szoftvertervezésben használják ezeket a mintákat, melyekkel általános problémákat oldanak meg. Mondhatni olyanok, mint egy tervrajz, melyet mindig az adott kód tervezésére vonatkozóan átalakítunk. Gyakran összetévesztik az algoritmusokkal, azonban itt nem már előre meghatározott lépésekkel dolgozunk. Ez egy sokkal magasabb szintű leírása a megoldásnak.

Tervezési minták összetevői:

- **szándék:** A problémát és a megoldást egyszerre írja le.
- **motiváció:** Részletesebben kifejti a problémát, valamint a kiválasztott minta milyen megoldást kínál.
- **struktúra:** Az osztályokat, azok részeit és az ezek közti kapcsolatokat mutatja be.
- **Kód példa:** a választott programozási nyelvben megkönnyíti a minta mögött meghúzódó gondolat átlátását, megértését.

Történetük:

Konkrétan nem tudjuk megmondani, ki találta fel őket, ugyanis ezek nem homályos, kifinomult fogalmak. Amikor egy adott probléma leírására folyamatosan az adott módszert használták, valaki adott neki egy nevet (mi emberek már csak ilyenek vagyunk).

A tervezési minták koncepcióját először Christopher Alexander vetette fel A *pattern Language: Towns, Buildings, Construction* című könyvében. Lényegében ez a városi környezet „nyelvét” hivatott leírni. (Ha belegondolunk ez könnyen implementálható kódban is.)

Az alapötletet négy szerző, név szerint Erich Gamma, John Vlissides, Ralph Johnson, and Richard Helm vetették fel 1994-ben. A négy szerző publikált egy könyvet *Design Patterns: Elements of Reusable Object-Oriented Software* címen. A könyv 23 mintát adott meg különböző problémák megoldására. Fontos, hogy ezek az objektum-orientált környezethez

biztosítanak megoldási lehetőségeket! (Azóta természetesen már nem csak az objektum-orientált programozáshoz jelentek meg tervezési minták.)

Miért érdemes őket tanulni?

Könnyen előfordulhat, hogy bizonyos mintákat mi is alkalmazunk a kódolásunk során, csak ennek nem vagyunk tudatában. Azonban érdemes ezeket megtanulni, ugyanis ezek a minták már kiállták az idő vasfogát, azaz már sokat próbált módszerek a szoftvertervezésben. A tervezési minták egy közös nyelvet adhatnak a fejlesztőcsapat(unk)nak. Ez megkönnyítheti a csapatmunkát.

Az emberek mikkel kritizálják?

Mint minden létező dolgot, a tervezési minták is rengeteg kritikát kaptak az évek során. Például, ha a kiválasztott nyelv nem rendelkezik a megfelelő szintű absztrakcióval, akkor a minta szinte már „szuper képességeket” ad a nyelvnek. Számomra viszont ez felveti a kérdést: Ha valami egyszerűbbé teszi az életünket, akkor mi a baj vele?

Néhányan esetleg teljesen implementálhatják a projektjükre az adott mintát, holott változtatni kellene rajta. Mivel azonban ezt nem vették észre, így nem tekintik hatékonynak a mintákat.

Néhányan akkor is a mintákat alkalmazzák, amikor elég lenne egy szimpla kód is.

Kategorizálásuk:

Bonyolultságukban, részletezettségükben és abban különböznek, hogy milyen mértékben alkalmazhatók egy egész rendszer tervezésére.

idiomok: A legegyszerűbb, alacsony szintű minták. Egyetlen programozási nyelvre használhatóak.

építészeti minták: Gyakorlatilag bármilyen programozási nyelvhez alkalmazhatóak. Egy teljes alkalmazás architektúráját le lehet velük írni.

cél szerint:

- **létrehozási minták:** Objektum létrehozási mechanikák, melyek a rugalmasságot és a kód újrahasználatosságát biztosítják.
- **strukturális minták:** Elmagyarázzák, hogyan építsünk fel objektumokat és osztályokat nagyobb struktúrákba, mialatt ugyanúgy megmarad a rugalmasságuk.
- **viselkedési minták:** A hatékony kommunikációról és az adott objektumok felelősségeiről gondoskodnak.

Tervezési minták kategóriák szerinti felsorolása:

létrehozási minták:

- **factory method:** Egy interfészt biztosít objektumok létrehozására a superclassban, de lehetővé teszi az alosztályok számára, hogy megváltoztassák a létrehozandó objektumok típusát.
- **abstract factory:** Összefüggő objektumok családjainak létrehozását biztosítja, konkrét osztályainak specifikálása nélkül.
- **builder:** Komplex objektumok létrehozását biztosítja, lépésről-lépésre. A minta megengedi, hogy különböző típusokat, reprezentációkat hozzunk létre egy objektumról ugyanazzal a konstrukciós kóddal.
- **prototípus:** Létező objektumok másolását teszi lehetővé osztályaiktól való függések nélkül.
- **singleton:** Lehetővé teszi, hogy biztosítsuk, hogy egy objektumnak egy instance-e legyen anélkül, hogy globális hozzáférési pontot adnánk hozzá.

strukturális minták:

- **adapter:** Olyan objektumoknak, melyek inkompatibilis interfészekkel rendelkeznek, együttműködést biztosítanak.
- **bridge:** Megengedi, hogy egy nagyobb osztály vagy egy adag közeli kapcsolattal rendelkező osztályt két különböző hierarchiába szervezzük. Ez az implementáció és az absztrakció, melyek egymástól függetlenül is fejleszthetőek.

- **composite:** Lehetővé teszi, hogy objektumokat fa struktúrákba rendezzünk, majd ezekkel a struktúrákkal úgy dolgozzunk, mintha egyedi objektumok lennének.
- **decorator:** Megengedi, hogy új viselkedéseket adjunk az objektumoknak oly módon, hogy ezeket az objektumokat speciális csomagoló objektumokba rakjuk, melyek az új viselkedéseket tartalmazza.
- **facade:** Egy egyszerűsített interfész egy könyvtárhoz, egy keretrendszerhez, vagy bármilyen másik komplex osztályok gyűjteményéhez.
- **flyweight:** Abban segít, hogy az elérhető RAM-ba több objektumot tudjunk rakni úgy, hogy az állapot közös részeit megosztjuk több objektum között, ahelyett, hogy az összes adatot minden egyes objektumban tárolnánk.
- **proxy:** Lehetővé teszi, hogy egy helyettesítő vagy egy helykitöltőt rakjunk egy másik objektum helyére. Egy proxy vezérli a hozzáférést az eredeti objektumhoz, lehetővé téve, hogy a kérések feldolgozása előtt vagy után valamilyen műveletet végezzünk.

viselkedési minták:

- **chain of responsibility:** Lehetővé teszi, hogy a kéréseket egy kezelők láncán továbbítsuk. Amikor egy kezelő megkap egy kérést, dönt arról, hogy feldolgozza azt, vagy továbbpasszolja a következő kezelőnek a láncban.
- **command:** Egy kérést egy különálló objektummá változtatja, mely tartalmaz minden információt a kérésről. Az átalakítás megengedi, hogy úgy küldjünk felkéréseket, hogy azok metódus argumentumként, késleltetésként vagy egy sorként küldjük el a végrehajtónak és megcsinálhatatlan feladatok elvégzését is lehetővé tegyünk.
- **iterator:** Lehetővé teszi, hogy egy gyűjtemény elemein keresztül haladjunk anélkül, hogy felfednénk a mögöttes reprezentációját (lista, verem, fa stb.).
- **mediator:** Lehetővé teszi, hogy csökkentsük az objektumok közötti kaotikus függőségeket. A minta korlátozza az objektumok közötti közvetlen kommunikációt, és arra kényszeríti őket, hogy csak egy közvetítő objektumon keresztül működjenek együtt.

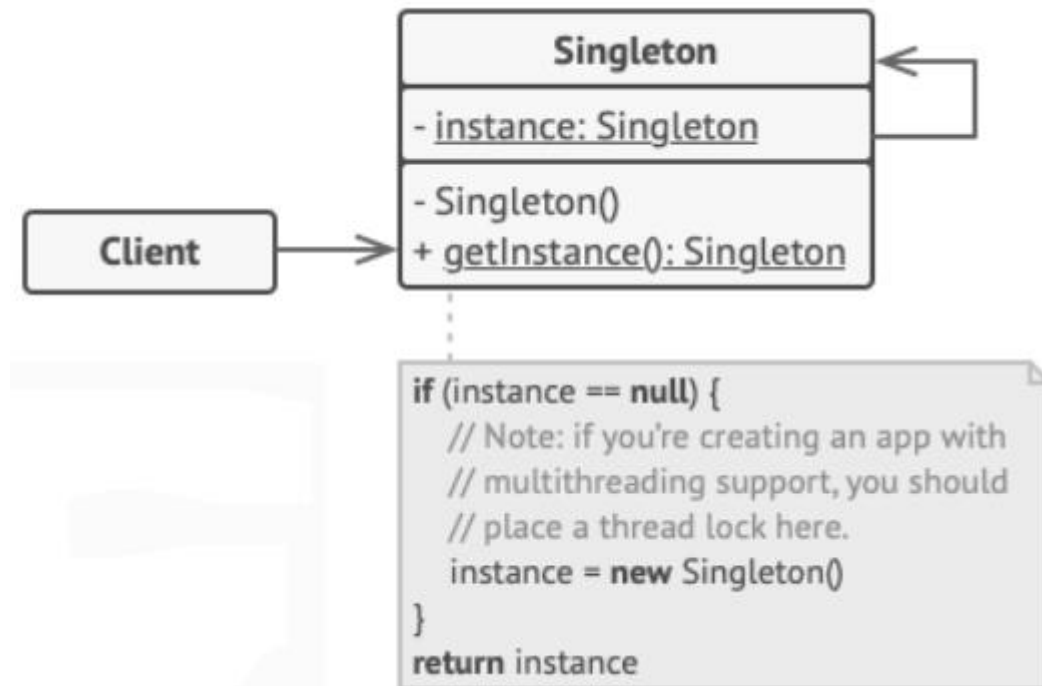
- **memento:** Objektumok előző állapotainak elmentését és visszaállítását teszi lehetővé anélkül, hogy felfednénk az eredeti objektum implementálásának részleteit.
- **observer:** Egy feliratkozási mechanizmust biztosít, hogy értesítsünk különböző objektumokat bármilyen eseményről, amely a vizsgált objektummal fog történni.
- **state:** Egy objektum viselkedésének megváltoztatását engedélyezi belső állapotváltozáskor. Úgy jelenik meg, mintha az objektum megváltoztatná az osztályát.
- **strategy:** Algoritmusok családját hozhatjuk vele létre és mindegyiket egy külön osztályba rakhatjuk, objektumaikat pedig felcserélhetővé tehetjük.
- **template method:** A superclass egy algoritmusának vázát írhatjuk meg benne, de a subclassok override-olhatják az algoritmus konkrét lépéseit a struktúrájának megváltoztatása nélkül.
- **visitor:** Lehetővé teszi, hogy az algoritmusokat elválasszuk azoktól az objektumoktól, amelyeken működnek.

Néhány tervezési minta részletes kifejtése

Singleton:

- Olyan létrehozási minta, mely lehetővé teszi, hogy biztosítsuk, hogy egy objektumnak egy instance-e legyen anélkül, hogy globális hozzáférési pontot adnánk hozzá.
- A Singleton osztály egy getInstance metódust generál, mely saját osztályának instance-át adja vissza.
- Konstruktort el kell rejtetni a kliens kód elől!
- Kizárólag csak a getInstance metódussal lehet megkapni a Singleton objektumot.
- A Singleton azért single (azaz egy), mert csak egyetlen egy instance érhető el a klienseknek.

- Minden más formáját letiltja az osztály létrehozásának, a speciális létrehozási metóduson kívül. Ez a metódus vagy az objektum létrehozását jelenti vagy már egy meglevő visszaadását.
- Leginkább akkor érdemes használni, ha a globális változókra szigorú megkötéseket szeretnénk bevezetni.



- Implementációja:
 1. Adjunk hozzá egy privát statikus mezőt az osztályhoz az egyetlen példány (singleton) tárolására.
 2. Deklaráljunk egy publikus statikus létrehozó metódust az egyetlen példány (singleton) lekérésére.
 3. Valósítsuk meg a "lazy initialization"-t úgy, hogy az első híváskor létrehoz egy objektumot és eltárolja a statikus mezőben. Később mindig ezt az egy példányt adja vissza.
 4. Tegyük az osztály konstruktorát priváttá. Így a statikus metódus még mindig tudja hívni a konstruktort, de más objektumok nem férnek hozzá.
 5. Cseréljük le az összes közvetlen hívást a singleton konstruktorára úgy, hogy a statikus létrehozó metódust használjuk helyette.
- A Facade osztályt gyakran átalakíthatjuk Singletonná, mivel a legtöbb esetben elegendő egyetlen facade objektum.

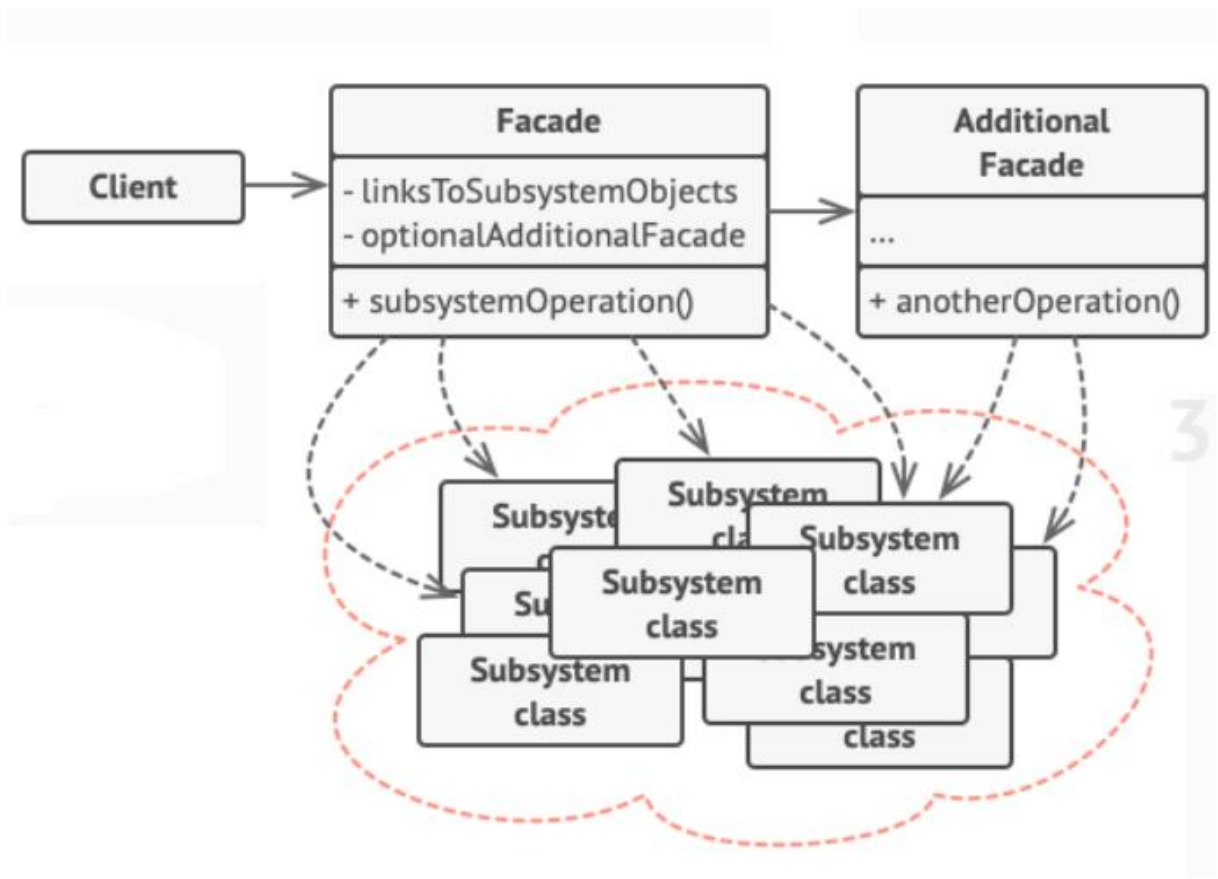
- A Flyweight hasonlíthatna a Singletonra, ha az összes megosztott állapotot egyetlen Flyweight objektumra tudnánk redukálni. Azonban két alapvető különbség van a minták között:

A Singletonból csak egy példány lehet, míg a Flyweight osztály több példányt is tartalmazhat eltérő belső állapotokkal.

A Singleton objektum lehet módosítható, míg a Flyweight objektumok változatlanok (immutable).

Facade:

- Olyan strukturális minta, mely egy egyszerűsített interfészt tartalmaz egy könyvtárhoz, egy keretrendszerhez, vagy bármilyen másik komplex osztályok gyűjteményéhez.
- Kényelmes hozzáférést biztosít egy bizonyos alrendszer részéhez.
- Tudja, hová kell a kliens kéréseit irányítani és hogyan kezelje a mozgó részeket.
- Egy további Facade osztályt is létrehozhatunk, hogy elkerüljük egy facade „beszennyezését” nem kapcsolódó funkcióval, mely így egy komplexebb struktúra létrejöttét okozná. Ezek a további Facade osztályok kliensek és további facade osztályok által egyaránt.
- A komplex alrendszer rengeteg különböző objektumból áll. Hogy értelmet adjunk nekik, ahhoz az implementációs részletekben át kell fésülnünk az például inicializációs objektumokat, mindezt a helyes sorrendben, továbbá megfelelő formátumú adatokkal kell ellátnunk őket.
- A kliens használja facade-ot ahelyett, hogy az alrendszer objektumait hívná meg közvetlenül.
- Ahelyett, hogy közvetlenül dolgoznánk a keretrendszer osztályaival, létrehozunk egy facade osztályt, amely elrejti ezeket a funkciókat a többi kód előtt. Ez a struktúra segít minimalizálni a keretrendszer jövőbeli verziókra történő frissítésének vagy másokra cserélésének erőfeszítéseit. Az alkalmazásban csak a facade metódusainak megvalósítását kell módosítanunk.



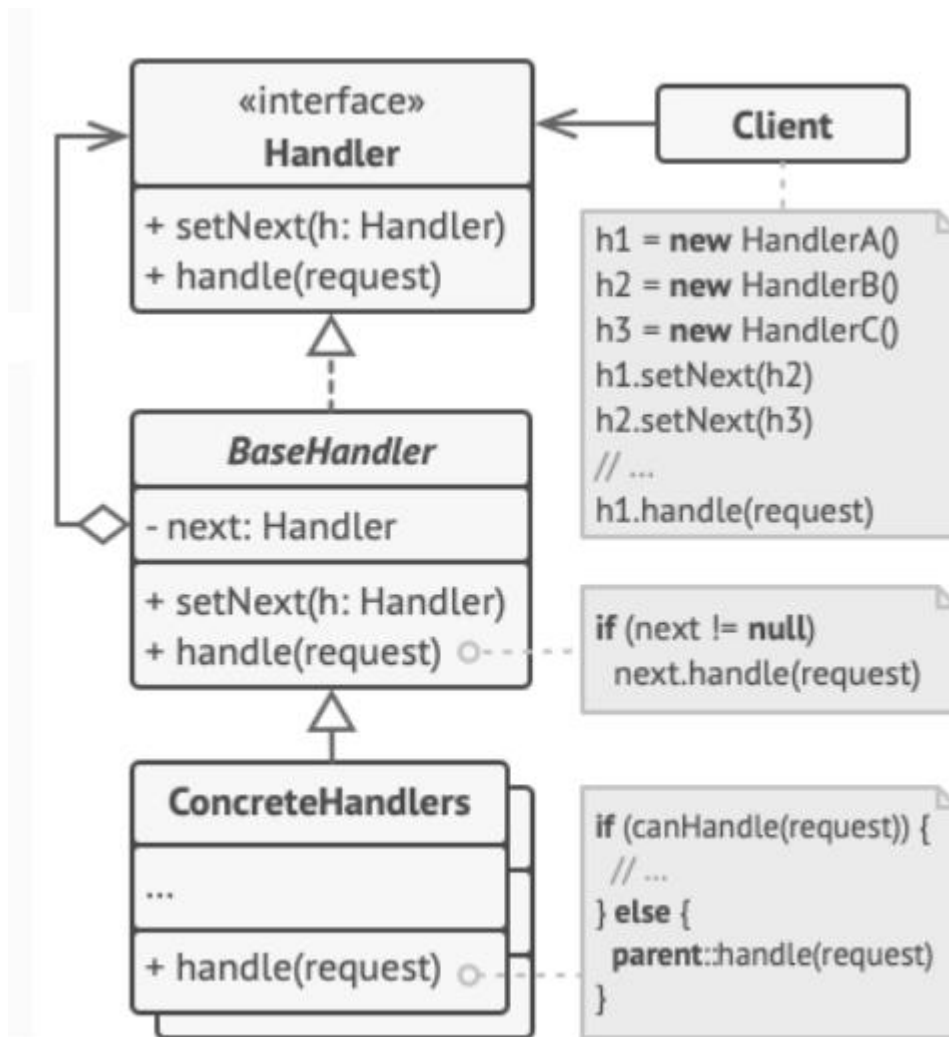
- Akkor érdemes használni a Facade mintát, ha limitált, de egyértelmű interfészeket szeretnénk egy komplex alrendszerhez.
- Az alrendszerek idővel egyre komplexebbé válhatnak. Még a tervezési minták használata is olykor több osztály létrehozásához vezethet. Bár rugalmasabbá válhatnak az alrendszerek, így is a konfigurációk és a sablonkódok így is növelik a klienssel szemben támasztott igényeket. A Facade ezt hivatott megoldani azzal, hogy egy „rövidítést” biztosít az alrendszer leghasználtabb funkcióihoz.
- Akkor is érdemes lehet használni, ha egy alrendszert különböző rétegekre szeretnénk bontani.
- Implementálása:
 1. Ellenőrizzük, hogy lehetséges-e egy egyszerűbb interfészt biztosítani az eredeti alrendszerénél! Ha függetlenné válik az interfész a kliens kódtól, akkor jó úton járunk.
 2. Deklaráljuk és implementáljuk ezt az interfészt egy új facade osztályba! Ennek az lesz a feladata, hogy a hívásokat átirányítja a kliens kódtól az alrendszer megfelelő objektumaihoz. A facade-nak az alrendszer inicializációjáért és a további életciklusának fenntartásáért amíg a kliens kód meg nem teszi ezt.

3. Hogy teljes profitot szerezzünk a tervezési mintából, az összes kliens kódnak a facadeon keresztül kell kommunikálnia az alrendszerrel.
 4. Ha túl nagygyá válik, akkor érdemes lehet megfontolni egy új, jobb kifinomultabb facade osztály létrehozását.
- A Facade új interfészt definiál a meglévő objektumokhoz, míg az Adapter megpróbálja az adott interfészt használhatóvá tenni. Az Adapter általában csak egy objektumot csomagol, míg a Facade egy teljes alrendszert kezel.
 - Az Abstract Factory alternatívaként szolgálhat a Facade helyett, amikor csak az alrendszer objektumainak létrehozásának módját akarjuk elrejtetni a kliens kód előtt.
 - A Flyweight arra mutat példát, hogyan készíthetünk sok kis objektumot, míg a Facade arra, hogyan készíthetünk egyetlen objektumot, amely egy egész alrendszert képvisel.
 - A Facade és a Mediator hasonló feladatot látnak el: mindkettő segít szervezni a szoros összekapcsolódású osztályok együttműködését. A Facade egyszerűsített interfészt ad egy alrendszerhez, de nem vezet be új funkciókat, és az alrendszer nem tud a facade létezéséről. A Mediator központilag irányítja a kommunikációt a rendszer komponensei között, ahol a komponensek csak a mediátor objektumot ismerik.
 - A Facade gyakran átalakítható Singleton-ná, mivel a legtöbb esetben egyetlen facade objektum elegendő.
 - A Facade és a Proxy hasonlóak abban, hogy mindkettő egy komplex entitást burokbá helyez és inicializálja azt. A Proxy azonban ugyanazt az interfészt használja, mint a szolgáltató objektum, így cserélhető vele, míg a Facade nem.

Chain of responsibility:

- Olyan létrehozási minta, mely lehetővé teszi, hogy a kéréseket egy kezelők láncán továbbítsuk. Amikor egy kezelő megkap egy kérést, dönt arról, hogy feldolgozza azt, vagy továbbpasszolja a következő kezelőnek a láncban.

- A kezelő deklarál egy interfészt, mely minden konkrét kezelőben közös. Ez általában egyetlen metódust tartalmaz, azonban tartalmazhat a láncban következő kezelőre vonatkozóan is tartalmazhat egy metódust.
- Az alap kezelő egy válaszható osztály, melybe a sablonkódot rakhatjuk, ami minden kezelőre igaz. Általában ez az osztály egy referenciát tartalmaz a következő kezelőre. A kliensek építhetnek egy láncot a következő kezelő továbbadásával a konstruktor számára vagy az előző setterével. Az osztály implementálhat egy alap kezelői viselkedést, azaz a következő kezelőnek adhatja a futtatást, miután leellenőrizte, hogy az létezik-e.
- A kliens egyszer vagy dinamikusan is állíthat össze láncokat, alkalmazhatósági logikájának függvényében. Vegyük azt számításba, hogy a lánc bármely kezelőjének küldhetünk utasítást, nem csak az elsőnek.
- A konkrét kezelők tartalmazzák a tényleges kódot a kérések feldolgozására.
- A kérés érkezése után minden kezelőnek el kell döntenie, hogy feldolgozza-e, esetleg dobja-e tovább a láncban vagy sem.
- A kezelők általában önállóak és változtathatatlanok, azaz csak egyszer, a konstruktorukon keresztül fogadják el az összes szükséges adatot.



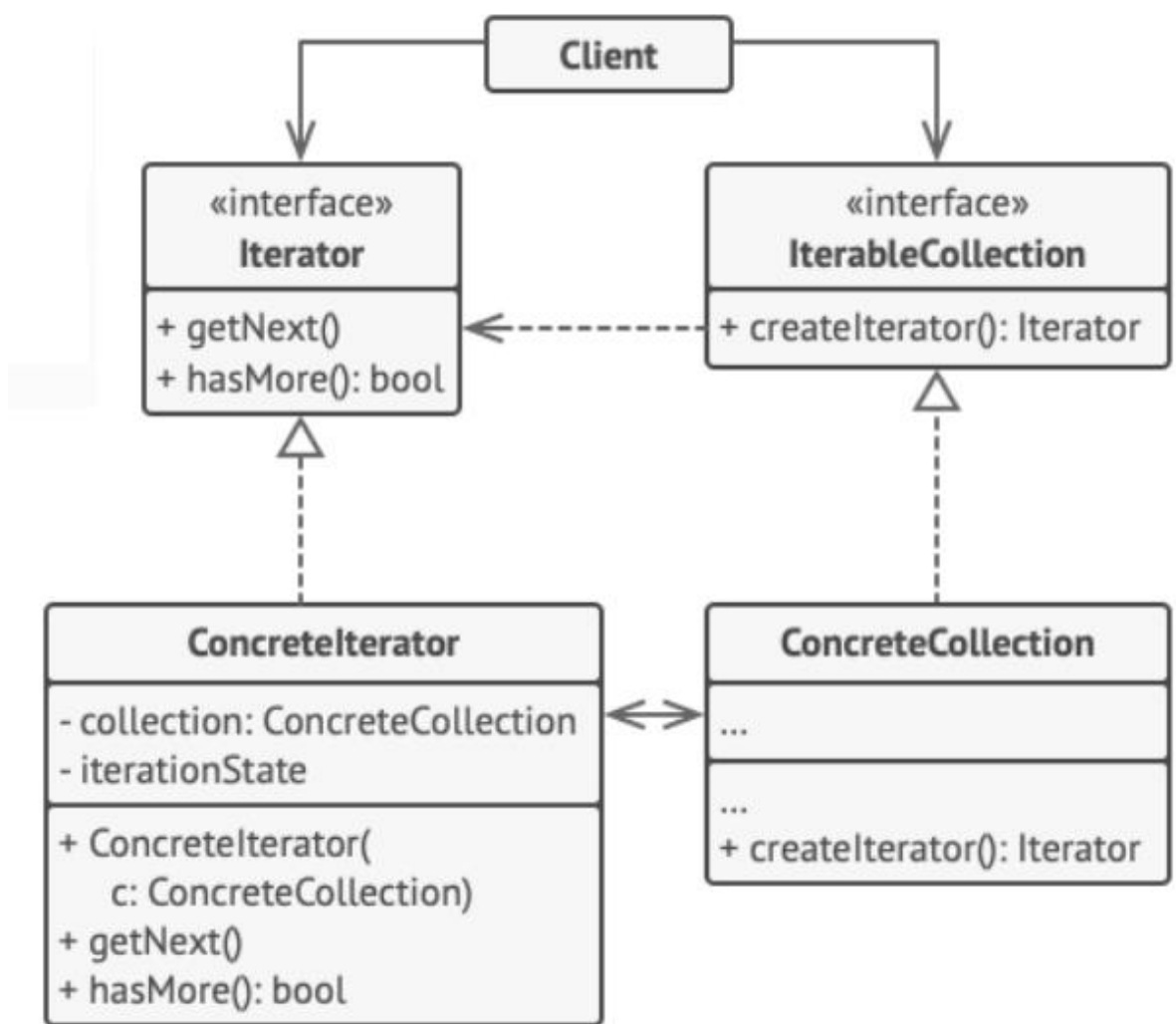
- Használjunk **Chain of Responsibility** mintát, amikor a programunk különböző típusú kéréseket vár, amelyeket különböző módokon kell feldolgozni, de a pontos kéréstípusok és azok sorrendje előre nem ismert.
- Több kezelőt is összefűzhetünk egy láncba, és amikor kapunk egy kérést, akkor lényegében minden kezelő eldönti, hogy az ő feladata-e.
- Akkor is alkalmazhatjuk, ha szükséges, hogy különböző kezelőket egy meghatározott sorrendben futtassunk.
- Akkor is jól jöhet, ha a futás ideje alatt kell a kezelők futásának sorrendjét megváltoztatni.
- Implementálása:
 1. Deklaráljuk a kezelő interfészt, és írjuk le egy módszer aláírását, amely a kérések kezelésére szolgál. Döntsük el, hogyan fogja az ügyfél átadni a kérés adatokat a módszernek. A legrugalmasabb módja ennek az, ha a kérést egy objektummá alakítjuk, és argumentumként átadjuk a kezelő módszernek.

2. A sablonok duplikálásának elkerülése végett érdemes lehet egy absztrakt alap kezelőt létrehozni, a kezelő interfészből származtatva. Ez az osztály rendelkezzen egy mezővel, amely tárolja a következő kezelőre mutató hivatkozást a láncban. Fontolóra kell vennünk az osztály immutabilis (nem módosítható) megvalósítását. Azonban, ha a láncokat futásidőben módosítani szeretnénk, szükségünk lesz egy setter metódusra a referencia mező értékének módosításához. A kezelő módszer számára kényelmes alapértelmezett viselkedést is megvalósíthatunk, amely a kérést továbbítja a következő objektumnak, kivéve, ha már nincs több objektum a láncban. A konkrét kezelők ezt a viselkedést úgy használhatják, hogy meghívják a szülő metódust.
3. A konkrét kezelő egyenként hozhasson létre alosztályokat és implementálja a kezelési metódusait. El kell döntenie, hogy futtatja vagy továbbadja a kérést.
4. A kliens vagy magától építi fel a láncot vagy egy előre elkészített láncot kap más objektumokból. Utóbbi esetben néhány factory osztályt is implementálnunk kell a lánc felépítéséhez, konfigurációtól és környezettől függően.
5. A kliens bármely kezelőt elindíthatja a láncban, nem csak az elsőt. A kérés egészen addig fut a láncon, míg valamelyik kezelő el nem utasítja a továbbadást, vagy végig nem ment a láncon.
6. Dinamikus természetéből fakadóan a következő lehetőségekre készen kell állnia:
 - Egyetlen elemből állhat a lánc.
 - Néhány kérés nem éri el a lánc végét.
 - Mások kezeletlenül érhetik el a lánc végét.

Iterator:

- Olyan viselkedési minta, mely lehetővé teszi, hogy egy gyűjtemény elemein keresztül haladjunk anélkül, hogy felfednénk a mögöttes reprezentációját (lista, verem, fa stb.).
- Az iterátor interfész deklarálja azokat az operációkat, amelyek a gyűjtemény bejárásához szükségesek: a következő elem lekérése, az aktuális pozíció lekérdezése, az iteráció újraindítása stb.

- A konkrét iterátorok specifikus algoritmusokat implementálnak a gyűjtemény bejárására. Az iterátor objektumnak saját magának kell nyomon követnie a bejárás előrehaladását. Ez teszi azt lehetővé, hogy több iterátor egymástól függetlenül bejárja ugyanazt a gyűjteményt.
- A Collection interfész egy vagy több metódust deklarál az iterátorok lekérésére, amelyek kompatibilisek a gyűjteménnyel. Fontos megjegyezni, hogy a metódusok visszatérési típusának iterátor interfészként kell szerepelnie, hogy a konkrét gyűjtemények különböző típusú iterátorokat adhatjanak vissza.
- A konkrét gyűjtemények minden egyes alkalommal új példányokat adnak vissza egy adott konkrét iterátor osztályból, amikor a kliens kéri őket.
- A kliens mind a gyűjteményekkel, mind az iterátorokkal az interfészeiken keresztül dolgozik. Így a kliens nincs összekapcsolva konkrét osztályokkal, lehetővé téve, hogy különböző gyűjteményeket és iterátorokat használjon ugyanazzal a kliens kóddal.
- Jellemzően a kliensek nem hozzák létre saját iterátoraikat, hanem a gyűjteményektől kéri őket. Azonban bizonyos esetekben a kliens közvetlenül is létrehozhat egyet; például, amikor a kliens saját speciális iterátort definiál.



- Használjuk akkor, amikor a gyűjteményünk komplex adatszerkezettel rendelkezik, de ezt a kliensek elől el akarjuk rejtetni.
- Az iterátor a bonyolult adatstruktúrával való munkát becsomagolja, és a kliens számára egyszerű módszereket kínál a gyűjtemény elemeinek elérésére. Bár ez a megoldás kényelmes a kliens számára, ugyanakkor védi a gyűjteményt a gondatlan vagy rosszindulatú műveletektől, amelyeket a kliens közvetlenül a gyűjteménnyel dolgozva végrehajthatna.
- A bejáró kód ismétlődésének csökkentésére is használhatjuk az alkalmazásunkon belül.
- A kód nem egyértelmű iterációs algoritmusai hajlamosak túl hosszúvá válni. Az eredeti kód felelősségét elhomályosíthatja, kevésbé fenntarthatóvá téve ezzel. Az átmeneti kódok átvitele a kijelölt iterátorokba segíthet kódunk tisztává tételében.
- Ha különböző adatstruktúrákat szeretnénk átvinni, esetleg típusuk ismeretlen, akkor is segítségünkre lehet ez a minta.

- A minta néhány általános interfészt biztosít mind a gyűjtemények, mind az iterátorok számára. Mivel a kódunk innentől ezeket az interfészeket használja, továbbra is működni fog, ha különböző típusú gyűjteményeket és iterátorokat adunk át, amelyek implementálják ezeket az interfészeket.
- Implementálása:
 1. Deklaráljuk az iterátor interfészt! Legalább egy metódussal rendelkeznie kell, hogy fetchelhessük a következő elemet a gyűjteményből. A kényelem érdekében több metódust is hozzáadhatunk, mondjuk a következő elem lekérését, az aktuális pozíció nyomon követését és az iteráció végét ellenőrző metódust
 2. Deklaráljuk a gyűjtemény interfészt, és írjuk le a metódust az iterátorok lekérésére. A visszatérési típusnak meg kell egyeznie az iterátor interfész típusával. Létrehozhatunk hasonló metódusokat is, ha több különböző iterátor csoportot szeretnénk definiálni.
 3. Implementáljunk konkrét iterátor osztályokat azokhoz a gyűjteményekhez, melyeket bejárhatóvá szeretnénk tenni iterátorokkal. Az iterátor objektumnak egyetlen gyűjteménybeli instance-hoz kell lennie láncolva. Általában ez a láncolat az iterátor konstruktorán keresztül jön létre.
 4. Implementáljuk a gyűjtemény interfészt a gyűjtemény osztályainkban! Az alap ötlet az, hogy a kliens számára biztosítsunk egy rövidítést az iterátorok létrehozásához, egy specifikus gyűjtemény osztályhoz igazítva. A gyűjtemény objektum át kell, hogy adja magát az iterátor konstruktorának, hogy létrehozhassa a kapcsolatot köztük.
 5. Nézzük át a kliens kódot, és cseréljük le az összes gyűjtemény bejárására vonatkozó kódot iterátorok használatára. A kliens minden alkalommal új iterátor objektumot kér, amikor a gyűjtemény elemeit szeretné bejárni.
- Használhatjuk az iterátorokat a Kompozit fák bejárására.
- Használhatjuk a Factory Method mintát az iterátorral együtt, hogy a gyűjtemény alosztályai különböző típusú iterátorokat adjanak vissza, amelyek kompatibilisek a gyűjteményekkel.

- Használhatjuk a Memento mintát az iterátorral együtt, hogy rögzítsük az aktuális iteráció állapotát, és szükség esetén visszaállítsuk azt.
- Használhatjuk a Visitor mintát az iterátorral együtt, hogy bejárjunk egy bonyolult adatstruktúrát, és műveleteket hajtsunk végre annak elemein, még akkor is, ha azok mind különböző osztályokból származnak.